

Department of computer science and engineering
Programming With Python
Project Report
CSE AI 1 Group 1
Academic session 2025-29



Indira Gandhi Delhi Technical University for Women

Project File:
Python 10-in-1 Game Collection

Submitted To:
Dr. Dipty Tripathi

Submitted By:
Ananya Joshi
02101172025

Contents

1	Introduction	3
2	System/Software Requirements	3
2.1	Hardware Requirements.....	3
2.2	Software Requirements	3
3	System Design Workflow	4
4	Project Code	6
4.1	Imports and Global Variables	6
4.2	Game 1: FLAMES Compatibility.....	6
4.3	Game 2: Guessing Number	8
4.4	Game 3: Rock Paper Scissors	8
4.5	Game 4: Blackjack	9
4.6	Utility 5: Password Generator	11
4.7	Game 6: Magic 8-Ball.....	11
4.8	Game 7: Word Scramble.....	12
4.9	Game 8: Mad Libs.....	13
4.10	Game 9: Dice Rolling Simulator	14
4.11	Game 10: Pet Simulator	15
5	Snapshot of Outputs/Results	17
5.1	Main Menu	17
5.2	Game 1: FLAMES Compatibility.....	17
5.3	Game 2: Guessing Number Game	18
5.4	Game 3: Rock Paper Scissors	18
5.5	Game 4: Blackjack	19
5.6	Utility 5: Password Generator	19
5.7	Game 6: Magic 8-Ball.....	19
5.8	Game 7: Word Scramble.....	19
5.9	Game 8: Mad Libs.....	20
5.10	Game 9: Dice Rolling Simulator	20
5.11	Game 10: Pet Simulator	21

1. Introduction

This project presents a "10-in-1 Game Collection" developed using the Python programming language. It is a console-based application that bundles ten different mini-games and utilities into a single, cohesive, menu-driven program.

The primary objective of this project is to demonstrate foundational programming concepts in Python, including function definition, control flow (loops and conditionals), data structures (lists and dictionaries), user input handling, and the use of standard libraries like `random`, `time`, and `sys`.

The collection includes a diverse set of programs:

- **Classic Games:** Rock Paper Scissors, Guessing Number, Blackjack.
- **Fun Tools:** FLAMES Compatibility, Magic 8-Ball, Mad Libs.
- **Simulations:** Pet Simulator, Dice Roller.
- **Skill Games:** Word Scramble.
- **Utilities:** Password Generator.

This report documents the system requirements, design workflow, complete source code, and sample outputs of the project.

2. System/Software Requirements

The project is lightweight and relies only on the Python Standard Library, making it highly portable.

2.1 Hardware Requirements

- **Processor:** Any modern CPU (x86, x64, ARM).
- **RAM:** > 128 MB of free RAM.
- **Storage:** < 10 MB of free disk space for the .py file.

2.2 Software Requirements

- **Operating System:** Any OS that can run a Python interpreter, such as Windows 10/11, macOS, or any Linux distribution (e.g., Ubuntu, Fedora).
- **Core Software:** **Python 3.x** (version 3.6 or newer is recommended).
- **Required Libraries:** All imported modules (`random`, `time`, `sys`, `os`) are part of the Python Standard Library and do not require separate installation via `pip`.
- **Runtime Environment:** A command-line terminal (e.g., Command Prompt, PowerShell, Terminal, or an IDE's built-in console).

3. System Design Workflow

The application is built on a modular, menu-driven architecture. The entire workflow is controlled by a central function, `main_menu()`.

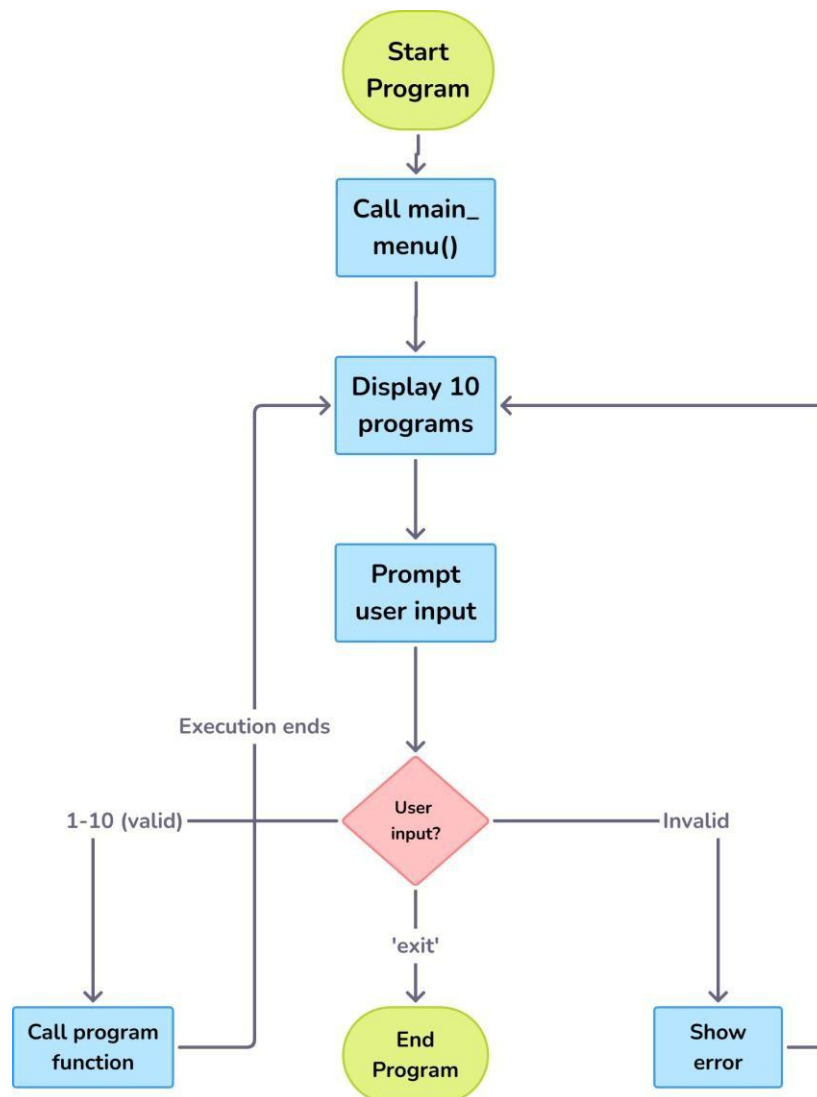


Figure 1: Workflow of the Program

1. **Initialization:** The program starts when the `if _name__ == "__main__":` block is executed, which calls `main_menu()`.
2. **Main Loop:** The `main_menu()` function contains a `while True:` loop, which continuously displays the list of 10 available programs to the user.
3. **User Input:** The program prompts the user to enter a choice (a number from 1 to 10) or 'exit'.
4. **Routing:** An `if/elif/else` block evaluates the user's choice.

- If a valid number (1-10) is chosen, the corresponding function (e.g., `flames_game()`, `blackjack_game()`) is called.
 - If 'exit' is chosen, the `sys.exit()` command is called, and the program terminates.
 - If the input is invalid, an error message is shown, and the loop repeats, displaying the menu again.
5. **Program Execution:** The called game/utility function (e.g., `pet_simulator()`) takes control. It runs its own internal logic and loops until the user finishes the game or chooses to quit that specific game.
 6. **Return to Menu:** Once the specific game function finishes, control returns to the `main_menu()` loop, which displays the menu again for the user to make a new selection.

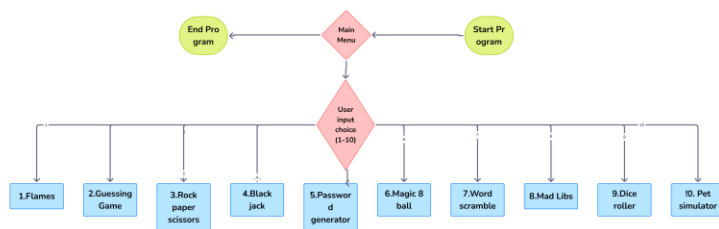


Figure 2: Flowchart showing the different components of the program

The code for this central workflow controller is as follows:

```

1 # =====
2 # MASTER MENU (The entry point for the project)
3 # =====
4
5 def main_menu():
6     while True:
7         print("\n" + "#" * 50)
8         print("WELCOME TO THE PYTHON 10-IN-1 GAME COLLECTION!")
9         print("#" * 50)
10        print("Please choose a program to run:")
11        print("1. FLAMES Compatibility Calculator")
12        print("2. Guessing Number")
13        print("3. Rock Paper Scissors")
14        print("4. Blackjack (Simplified)")
15        print("5. Password Generator (Utility)")
16        print("6. Magic 8-Ball")
17        print("7. Word Scramble")
18        print("8. Mad Libs")
19        print("9. Dice Rolling Simulator")
20        print("10. Pet Simulator (State Management)")
21        print("Type 'exit' to quit the collection.")
22        print("-" * 50)
23
24        choice = input("Enter your choice (1-10 or exit): ").strip().
lower()

```

```

25
26     if choice == '1':
27         flames_game ()
28     elif choice == '2':
29         guessing_number ()
30     elif choice == '3':
31         rock_paper_scissors ()
32     elif choice == '4':
33         blackjack_game ()
34     elif choice == '5':
35         password_generator ()
36     elif choice == '6':
37         magic_8_ball ()
38     elif choice == '7':
39         word_scramble ()
40     elif choice == '8':
41         mad_libs ()
42     elif choice == '9':
43         dice_roller ()
44     elif choice == '10':
45         pet_simulator ()
46     elif choice == 'exit':
47         print("Thank you for using the Game Collection ! Goodbye.")
48         sys.exit ()
49     else :
50         print("Invalid choice . Please enter a number between 1 and
51         10 , or 'exit' .")
52 # Start the entire program
53 if __name__ == "__main__":
54     main_menu ()

```

4. Project Code

This section contains the complete source code for all modules and utilities used in the project.

4.1 Imports and Global Variables

These are the libraries imported at the start of the script, along with the global deck variable used by the Blackjack game.

```

1 import random
2 import time
3 import sys
4 import os # Necessary for the FLAMES game
5
6 # Global deck definition used by Blackjack
7 deck = {'2': 2, '3': 3, '4': 4, '5': 5, '6': 6, '7': 7, '8': 8, '9': 9,
8         '10': 10, 'Jack': 10, 'Queen': 10, 'King': 10, 'Ace': 11}

```

4.2 Game 1: FLAMES Compatibility

```

1 def flames_game():
2     """
3     Calculates the relationship between two names using the FLAMES
4     algorithm.
5     """
6     print("\n--- 1. FLAMES Compatibility Calculator ---")
7
8     # Clear the console before taking input (if possible)
9     if os.name == 'nt':
10         os.system('cls')
11     else:
12         os.system('clear')
13
14     n1 = input("Enter the first name: ")
15     n2 = input("Enter the second name: ")
16
17     # Clean the names: lowercase and remove spaces
18     name1 = n1.lower().replace(" ", "")
19     name2 = n2.lower().replace(" ", "")
20
21     flames_map = {
22         "F": "Friends",
23         "L": "Lovers",
24         "A": "Affection",
25         "M": "Marriage",
26         "E": "Enemies",
27         "S": "Siblings"
28     }
29
30     name1_list = list(name1)
31     name2_list = list(name2)
32
33     # Find and remove matching letters
34     for ch in name1[:]: # Iterate over a copy of name1
35         if ch in name2_list:
36             name1_list.remove(ch)
37             name2_list.remove(ch)
38
39     comb = name1_list + name2_list
40     n = len(comb) # Total number of remaining letters
41
42     # FLAMES calculation loop (Josephus problem variant)
43     c = ['F', 'L', 'A', 'M', 'E', 'S']
44     idx = 0
45     while len(c) > 1:
46         # Calculate the index to remove (n steps around the list,
47         # adjusting for 0-index)
48         idx = (idx + n - 1) % len(c)
49         c.pop(idx)
50
51     result_letter = c[0]
52     result = flames_map[result_letter]
53
54     print("\nCalculating the FLAMES relationship ...")
55     time.sleep(1.5)
56     print(f"The number of remaining letters is: {n}")
57     print(f"The final FLAMES letter is: {result_letter}")
58     print(f"\t\t\t\t\tResult: {n1.title()} and {n2.title()} are **{

```

```

57     result}**!           ")
58     print(" -----")
59     time.sleep(3)
60     print("Returning to the main menu.....")

```

4.3 Game 2: Guessing Number

```

1  def guessing_number():
2      print("\n--- 2. Guessing Number Game ---")
3
4      top_of_range = input("Type a number for the upper limit: ")
5
6      if top_of_range.isdigit():
7          top_of_range = int(top_of_range)
8
9          if top_of_range <= 0:
10             print('Please type a number larger than 0 next time.')
11             return
12
13         else:
14             print('Please type a number next time.')
15             return
16
17         random_number = random.randint(0, top_of_range)
18         guesses = 0
19
20         while True:
21             guesses += 1
22             user_guess = input("Make a guess: ")
23
24             if user_guess.isdigit():
25                 user_guess = int(user_guess)
26             else:
27                 print('Please type a number next time.')
28                 continue
29
30             if user_guess == random_number:
31                 print("You got it!")
32                 break
33             elif user_guess > random_number:
34                 print("You were above the number!")
35             else:
36                 print("You were below the number!")
37
38         print(f"You got it in {guesses} guesses.")

```

4.4 Game 3: Rock Paper Scissors

```

1  def rock_paper_scissors():
2      print("\n--- 3. Rock Paper Scissors ---")
3      user_wins = 0
4      computer_wins = 0
5      options = ["rock", "paper", "scissors"]
6
7      while True:

```



```

8     user_input = input(" Type Rock /Paper/ Scissors or Q to quit: ").
lower()
9     if user_input == "q":
10         break
11
12     if user_input not in options:
13         continue
14
15     random_number = random.randint(0, 2)
16     computer_pick = options[ random_number]
17     print(" Computer picked", computer_pick + ".")
18
19     if user_input == computer_pick :
20         print(" It's a tie!")
21     elif (user_input == "rock" and computer_pick == "scissors") or \
22 \
23         (user_input == "paper" and computer_pick == "rock") or \
24         (user_input == "scissors" and computer_pick == "paper"):
25         print(" You won!")
26         user_wins += 1
27     else :
28         print(" You lost!")
29         computer_wins += 1
30
31     print(f"You won {user_wins} times.")
32     print(f"The computer won {computer_wins} times.")
33     print(" Goodbye !")

```

4.5 Game 4: Blackjack

```

1 def calculate_sum (hand):
2     current_sum = sum(deck[card] for card in hand)
3     num_aces = hand.count(' Ace ')
4     while current_sum > 21 and num_aces > 0:
5         current_sum -= 10
6         num_aces -= 1
7     return current_sum
8
9 def deal_card (current_deck , hand):
10     card = current_deck.pop()
11     hand.append(card)
12     return deck[card]
13
14 def blackjack_game():
15     print("\n--- 4. Blackjack (Simplified) ---")
16
17     print(" Welcome to the Game of Blackjack!")
18
19     current_deck = list(deck.keys()) * 4
20     random.shuffle(current_deck)
21
22     player_hand = []
23     dealer_hand = []
24
25     # Initial deal
26     deal_card(current_deck, player_hand)

```

```

27 deal_card(current_deck, dealer_hand)
28 deal_card(current_deck, player_hand)
29 deal_card(current_deck, dealer_hand)
30
31 player_sum = calculate_sum(player_hand)
32 deal_sum = calculate_sum(dealer_hand)
33
34 print(f"Dealer's face-up card is: {dealer_hand[0]}")
35 print(f"Your hand is: {' '.join(player_hand)} (Sum: {player_sum})\n")
36
37 # Player's turn
38 while player_sum < 21:
39     hit_stand = input("Do you want to hit or stand? (h/s): ")
40     if hit_stand.lower() == 'h':
41         deal_card(current_deck, player_hand)
42         player_sum = calculate_sum(player_hand)
43         print(f"You drew a {player_hand[-1]}.")
44         print(f"Your hand is now: {' '.join(player_hand)} (Sum: {player_sum})\n")
45     elif hit_stand.lower() == 's':
46         break
47     else:
48         print("Invalid input. Please enter 'h' or 's'.")
49
50 # Outcome after player's turn
51 if player_sum > 21:
52     print(f"Busted! Your sum is {player_sum}. You lose.")
53     return
54
55 # Dealer's turn
56 print(f"Dealer reveals their second card. Hand: {' '.join(dealer_hand)} (Sum: {deal_sum})")
57 while deal_sum < 17:
58     print("Dealer hits.")
59     deal_card(current_deck, dealer_hand)
60     deal_sum = calculate_sum(dealer_hand)
61     print(f"Dealer drew a {dealer_hand[-1]}.")
62     print(f"Dealer's hand is now: {' '.join(dealer_hand)} (Sum: {deal_sum})\n")
63
64 # Final outcome
65 print("--- Final Results ---")
66 print(f"Your final hand: {' '.join(player_hand)} (Sum: {player_sum})")
67 print(f"Dealer's final hand: {' '.join(dealer_hand)} (Sum: {deal_sum})")
68
69 if deal_sum > 21:
70     print("Dealer busted! You win!")
71 elif deal_sum > player_sum:
72     print("Dealer wins!")
73 elif player_sum > deal_sum:
74     print("You win!")
75 else:
76     print("It's a push (tie)!")

```

4.6 Utility 5: Password Generator

```
1 def password_generator():
2     print("\n--- 5. Password Generator ---")
3
4     try:
5         n_input = input("Enter the length of password you want (min 4):")
6         if not n_input.isdigit():
7             print("Invalid input. Please enter a number.")
8             return
9         n = int(n_input)
10    except ValueError:
11        print("Invalid input. Please enter a number.")
12        return
13
14    if n < 4:
15        print("Password length must be at least 4 to ensure complexity.")
16    return
17
18    uppercase = "ABCDEFGHIJKLMNOPQRSTUVWXYZ "
19    lowercase = "abcdefghijklmnopqrstuvwxyz "
20    digits = "0123456789 "
21    specials = "!@#$%^&*"
22
23    password = [
24        random.choice(uppercase),
25        random.choice(lowercase),
26        random.choice(digits),
27        random.choice(specials)
28    ]
29
30    all_chars = uppercase + lowercase + digits + specials
31    for i in range(n - 4):
32        password.append(random.choice(all_chars))
33
34    random.shuffle(password)
35    password = "".join(password)
36
37    print(f"Generated password: **{password}**")
```

4.7 Game 6: Magic 8-Ball

```
1 def magic_8_ball():
2     print("\n--- 6. Magic 8-Ball ---")
3     responses = ["It is certain.", "It is decidedly so.", "Without a",
4                 "doubt.", "Yes definitely.",
5                 "Reply hazy, try again.", "Ask again later.", "Better not tell",
6                 "you now.",
7                 "Don't count on it.", "My reply is no.", "Very doubtful."]
8
9     print("Ask any YES or NO question you have.")
10    while True:
```

```

11     question = input("\nYour Question (Type 'exit' to quit): ").
strip()
12
13     if question.lower() == 'exit':
14         print("Thank you for consulting the Magic 8-Ball. Goodbye!")
15     )
16         break
17
18     if not question:
19         print("Please enter a question.")
20         continue
21
22     print("Shaking the ball...")
23     time.sleep(1.5)
24
25     answer = random.choice(responses)
26     print(f"\n    The Magic 8-Ball says: **{answer}**")

```

4.8 Game 7: Word Scramble

```

1 def word_scramble():
2     print("\n--- 7. Word Scramble ---")
3     print("Unscramble the letters to form the correct word.")
4
5     word_list = ["python", "college", "project", "submit", "coding", "
semester"]
6
7     correct_word = random.choice(word_list)
8     word_chars = list(correct_word)
9     random.shuffle(word_chars)
10    jumbled_word = "".join(word_chars)
11
12    while jumbled_word == correct_word: # Ensures it's actually
scrambled
13        random.shuffle(word_chars)
14        jumbled_word = "".join(word_chars)
15
16    attempts = 0
17    max_attempts = 3
18
19    print("-" * 30)
20    print(f"Your jumbled word is: **{jumbled_word.upper()}**")
21    print("-" * 30)
22
23    while attempts < max_attempts:
24        guess = input(f"Attempt {attempts + 1} of {max_attempts}. Your
guess: ").lower().strip()
25        attempts += 1
26
27        if guess == correct_word:
28            print(f"\n    **CONGRATULATIONS!** The word was: {
correct_word.upper()}")
29            return
30
31        elif attempts < max_attempts:
32            print("    Incorrect. Keep trying!")

```

```

33
34     print(f"\n      GAME OVER . The correct word was: **{correct_word.
upper ()}**")

```

4.9 Game 8: Mad Libs

```

1 def mad_libs ():
2     print("\n--- 8. Mad Libs ---")
3     print("      Welcome to Mad Libs! Please provide the following
words:")
4
5     # 1. Gather all the required input from the user
6     noun_1 = input("Enter a Noun (person , place , or thing): ").strip ()
7     adjective_1 = input("Enter an Adjective: ").strip ()
8     verb_past_tense = input("Enter a Verb in Past Tense: ").strip ()
9     adverb = input("Enter an Adverb (ends in -ly): ").strip ()
10    place = input("Enter a Name of a Place: ").strip ()
11    plural_noun = input("Enter a Plural Noun: ").strip ()
12    noun_2 = input("Enter another Noun: ").strip ()
13
14    print("\n--- Generating Your Story ---")
15    time.sleep (1.5)
16
17    # Story Templates
18    story_1 = f"A young {noun_1} was having a very {adjective_1} day at
the college . They suddenly {verb_past_tense} {adverb} across the
campus , heading straight for the {place} . They were trying to escape
from a horde of angry {plural_noun} , who wanted to steal their
precious **{noun_2}**! They knew they had to be fast to survive the
day."
19
20    story_2 = f" Last weekend , I decided to visit the famous { place } . I
packed a single , very **{ adjective_1 }** { noun_1 } and set off on my
journey . As soon as I arrived , I saw a crowd of wild **{plural_noun
}** running away from a tiny old {noun_2} . It seems the {plural_noun
} had been trying to steal the {noun_2} and failed! The tiny {noun_2
} then {verb_past_tense} **{adverb}** toward me , giving me a
triumphant wink before disappearing around a corner."
21
22    story_3 = f"During my first week at college , I discovered the
secret back entrance to the cafeteria , located right next to the **{
place}** . I was carrying a heavy , **{adjective_1}** **{noun_1}** and
trying to sneak past the guards . Suddenly , a huge stampede of **{
plural_noun}** came running at me! I quickly **{verb_past_tense}**
to the side and hid behind a giant **{noun_2}** . The cafeteria
manager looked at me **{adverb}** and simply shrugged , as if this
was a normal Tuesday . I realized college was going to be an
adventure !"
23
24    story_4 = f"It was a quiet afternoon in the main library at **{
place}** . I was studying my most **{adjective_1}** **{noun_1}** when
I noticed something strange . A flock of **{plural_noun}** were
gathered around a massive **{ noun_2 }** . The smallest { plural_noun }
suddenly **{ verb_past_tense }** into the air , holding a library book
titled 'The Art of Python' . The librarian looked up **{adverb}**
from her desk , sighed , and muttered : 'Not again !'"

```

```

25
26 all_stories = [story_1 , story_2 , story_3 , story_4 ]
27 final_story = random .choice (all_stories)
28
29 print("\n" + "=" * 40)
30 print("      ** Your Mad Libs Story **      ")
31 print("=" * 40)
32 print( final_story )
33 print("=" * 40)

```

4.10 Game 9: Dice Rolling Simulator

```

1 def dice_roller():
2     """
3     Simulates rolling one or more dice with a user-specified number of
4     sides.
5     """
6     print("\n--- 9. Dice Rolling Simulator ---")
7
8     # --- 1. Get Number of Dice ---
9     while True:
10         num_dice_input = input("How many dice do you want to roll (e.g
11         ., 2)? ").strip()
12
13         if num_dice_input.isdigit() and int(num_dice_input) > 0:
14             num_dice = int(num_dice_input)
15             break
16         else:
17             print("Invalid input. Please enter a positive number.")
18             continue
19
20     # --- 2. Get Number of Sides ---
21     while True:
22         num_sides_input = input("How many sides should the dice have (e
23         .g., 6, 10, 20)? ").strip()
24
25         if num_sides_input.isdigit() and int(num_sides_input) >= 2: #
26             Minimum 2 sides
27             num_sides = int(num_sides_input)
28             break
29         else:
30             print("Invalid input. Please enter a number greater than or
31             equal to 2.")
32             continue
33
34     results = []
35     total_sum = 0
36
37     print(f"\nRolling {num_dice} x D{num_sides} dice ...")
38
39     for i in range(num_dice):
40         roll = random.randint(1, num_sides)
41         results.append(str(roll))
42         total_sum += roll
43
44     print("-" * 40)

```

```

40 print(f"Roll Details: {num_dice} dice, {num_sides} sides each.")
41 print(f"Individual Rolls: {', '.join(results)}")
42 print(f"Total Sum: **{total_sum}**")
43 print("-" * 40)

```

4.11 Game 10: Pet Simulator

```

1 def pet_simulator():
2     """
3     Text-based pet simulation game.
4     """
5     print("\n--- 10. Pet Simulator (State Management) ---")
6
7     # --- 1. Initialize Pet Stats ---
8     hunger = 5
9     happiness = 5
10    energy = 5
11
12    # Set the boundaries
13    MAX_STAT = 20
14    MIN_STAT = 0
15
16    print("Take care of your new digital pet.")
17    print("Try to keep its stats from hitting 0!")
18    print("-" * 30)
19
20    # --- 2. Main Game Loop ---
21    while True:
22
23        # --- 3. Check Game Over Condition ---
24        if hunger <= MIN_STAT or happiness <= MIN_STAT or energy <=
MIN_STAT:
25            print("\nOh no! One of your pet's stats hit 0.")
26            if hunger <= MIN_STAT:
27                print("Your pet was too hungry. :(")
28            elif happiness <= MIN_STAT:
29                print("Your pet got too sad. :(")
30            else:
31                print("Your pet ran out of energy. :(")
32            print("--- Game Over ---")
33            break # Exit the while loop
34
35        # --- 4. Display Current Stats ---
36        print(f"\n--- Current Stats ---")
37        print(f"Hunger:      [{ '*' * hunger }{ ' ' * (MAX_STAT - hunger) }]
{hunger}/{MAX_STAT}")
38        print(f"Happiness: [{ '*' * happiness }{ ' ' * (MAX_STAT -
happiness) }] {happiness}/{MAX_STAT}")
39        print(f"Energy:      [{ '*' * energy }{ ' ' * (MAX_STAT - energy) }]
{energy}/{MAX_STAT}")
40        print("-" * 30)
41
42        # --- 5. Get Player Input ---
43        print("What would you like to do?")
44        print("    1. Feed (+4 Hunger)")
45        print("    2. Play (+4 Happiness)")

```

```

46 print("    3. Sleep (+4 Energy)")
47 print("    4. Quit (Return to menu)")
48 choice = input("Enter your choice (1-4): ")
49
50 # --- 6. Process Player's Choice ---
51 if choice == '1': # Feed
52     print("\nYou feed your pet. Yum!")
53     hunger = min(MAX_STAT, hunger + 4)
54
55 elif choice == '2': # Play
56     print("\nYou play with your pet. Fun!")
57     happiness = min(MAX_STAT, happiness + 4)
58
59 elif choice == '3': # Sleep
60     print("\nYou let your pet sleep. Zzzz ...")
61     energy = min(MAX_STAT, energy + 4)
62
63 elif choice == '4': # Quit
64     print("\nReturning to the main menu.")
65     break # Exit the while loop
66
67 else:
68     print("\nThat's not a valid choice. Try again.")
69     continue
70
71 # --- 7. Time Passes (Stats Decay) ---
72 print("... time passes ...")
73 hunger -= random.randint(1, 2)
74 happiness -= random.randint(1, 2)
75 energy -= random.randint(1, 2)
76
77 # Ensure stats don't go below the minimum
78 hunger = max(MIN_STAT, hunger)
79 happiness = max(MIN_STAT, happiness)
80 energy = max(MIN_STAT, energy)
81
82 # --- 8. Pause the Game ---
83 time.sleep(1)
84
85 print("\nGame session ended.")

```


5. Snapshot of Outputs/Results

5.1 Main Menu

This is the main entry point of the application, showing all 10 options.

```
#####  
WELCOME TO THE PYTHON 10-IN-1 GAME COLLECTION!  
#####  
Please choose a program to run:  
1. FLAMES Compatibility Calculator  
2. Guessing Number  
3. Rock Paper Scissors  
4. Blackjack (Simplified)  
5. Password Generator (Utility)  
6. Magic 8-Ball  
7. Word Scramble  
8. Mad Libs  
9. Dice Rolling Simulator  
10. Pet Simulator (State Management)  
Type 'exit' to quit the collection.  
-----  
Enter your choice (1-10 or exit): █
```

Figure 3: The main menu of the 10-in-1 Game Collection.

5.2 Game 1: FLAMES Compatibility

This shows the input prompts and the final result from the FLAMES calculator.

```
Enter the first name: riya  
Enter the second name: amit  
  
Calculating the FLAMES relationship...  
The number of remaining letters is: 4  
The final FLAMES letter is: E  
  
💖 Result: Riya and Amit are **Enemies**! 💖
```

Figure 4: FLAMES Compatibility

5.3 Game 2: Guessing Number Game

This shows the input prompts and the final result from the Guessing Number game.

```
--- 2. Guessing Number Game ---  
Type a number for the upper limit: 10  
Make a guess: 7  
You were above the number!  
Make a guess: 5  
You were above the number!  
Make a guess: 3  
You were below the number!  
Make a guess: 4  
You got it!  
You got it in 4 guesses.
```

Figure 5: Output of Guessing Number Game

5.4 Game 3: Rock Paper Scissors

This shows the input prompts and the final result from the Rock Paper Scissors.

```
--- 3. Rock Paper Scissors ---  
Type Rock/Paper/Scissors or Q to quit: Paper  
Computer picked rock.  
You won!  
Type Rock/Paper/Scissors or Q to quit: Q  
You won 1 times.  
The computer won 0 times.  
Goodbye!
```

Figure 6: Rock Paper Scissors

5.5 Game 4: Blackjack

This shows the input prompts and the final result from the Blackjack.

```
--- 4. Blackjack (Simplified) ---
Welcome to the Game of Blackjack!
Dealer's face-up card is: Ace
Your hand is: 3, 3 (Sum: 6)

Do you want to hit or stand? (h/s): h
You drew a 3.
Your hand is now: 3, 3, 3 (Sum: 9)

Do you want to hit or stand? (h/s): h
You drew a 8.
Your hand is now: 3, 3, 3, 8 (Sum: 17)

Do you want to hit or stand? (h/s): s
Dealer reveals their second card. Hand: Ace, King (Sum: 21)
--- Final Results ---
Your final hand: 3, 3, 3, 8 (Sum: 17)
Dealer's final hand: Ace, King (Sum: 21)
Dealer wins!
```

Figure 7: Gameplay of Blackjack

5.6 Utility 5: Password Generator

This shows the input prompts and the final result from the Password Generator.

```
--- 5. Password Generator ---
Enter the length of password you want (min 4): 12
Generated password: %gI%eI4hphzH
```

Figure 8: Output of Password Generator

5.7 Game 6: Magic 8-Ball

This shows the input prompts and the final result from the Magic 8-Ball.

```
--- 6. Magic 8-Ball ---
🔮 Ask any YES or NO question you have.

Your Question (Type 'exit' to quit): Will India win the next cricket world cup
Shaking the ball...

🌟 The Magic 8-Ball says: **Don't count on it.** 🌟
```

Figure 9: Gameplay of Magic 8-Ball

5.8 Game 7: Word Scramble

This shows the input prompts and the final result from the Word Scramble.

```

--- 7. Word Scramble ---
Unscramble the letters to form the correct word.
-----
Your jumbled word is: **TIUMSB**
-----
Attempt 1 of 3. Your guess: Submit

```

Figure 10: Output of Word Scramble

5.9 Game 8: Mad Libs

This shows the input prompts and the final result from the Mad Libs.

```

--- 8. Mad Libs ---
📌 Welcome to Mad Libs! Please provide the following words:
Enter a Noun (person, place, or thing): Ananya
Enter an Adjective: hardworking
Enter a Verb in Past Tense: had
Enter an Adverb (ends in -ly): smoothly
Enter a Name of a Place: delhi
Enter a Plural Noun: pens
Enter another Noun: cat

--- Generating Your Story ---

=====
💎 **Your Mad Libs Story** 💎
=====
It was a quiet afternoon in the main library at **delhi**. I was studying m
y most **hardworking** **Ananya** when I noticed something strange. A flock
of **pens** were gathered around a massive **cat**. The smallest pens sudde
nly **had** into the air, holding a library book titled 'The Art of Python'
. The librarian looked up **smoothly** from her desk, sighed, and muttered:
'Not again!'

```

Figure 11: Output of Mad Libs

5.10 Game 9: Dice Rolling Simulator

This shows the input prompts and the final result from the Dice Rolling Simulator.

```

--- 9. Dice Rolling Simulator ---
How many dice do you want to roll (e.g., 2)? 3
How many sides should the dice have (e.g., 6, 10, 20)? 10

Rolling 3 x D10 dice...
-----
Roll Details: 3 dice, 10 sides each.
Individual Rolls: 7, 3, 3
Total Sum: **13**
-----

```

Figure 12: Gameplay of Dice Rolling Simulator

5.11 Game 10: Pet Simulator

This shows the input prompts and the final result from the Pet Simulator.

```
--- 10. Pet Simulator (State Management) ---
Take care of your new digital pet.
Try to keep its stats from hitting 0!
-----

--- Current Stats ---
Hunger:  [*****] 5/20
Happiness: [*****] 5/20
Energy:   [*****] 5/20
-----
What would you like to do?
  1. Feed (+4 Hunger)
  2. Play (+4 Happiness)
  3. Sleep (+4 Energy)
  4. Quit (Return to menu)
Enter your choice (1-4): 1

You feed your pet. Yum!
...time passes...

--- Current Stats ---
Hunger:  [*****] 7/20
Happiness: [****] 4/20
Energy:   [***] 3/20
-----
What would you like to do?
  1. Feed (+4 Hunger)
  2. Play (+4 Happiness)
  3. Sleep (+4 Energy)
  4. Quit (Return to menu)
Enter your choice (1-4): 2

You play with your pet. Fun!
...time passes...

--- Current Stats ---
Hunger:  [*****] 6/20
Happiness: [*****] 7/20
Energy:   [*] 1/20
-----
What would you like to do?
  1. Feed (+4 Hunger)
  2. Play (+4 Happiness)
  3. Sleep (+4 Energy)
  4. Quit (Return to menu)
Enter your choice (1-4): 1

You feed your pet. Yum!
...time passes...

Oh no! One of your pet's stats hit 0.
Your pet ran out of energy. :(
--- Game Over ---
```

Figure 13: Gameplay of Pet Simulator